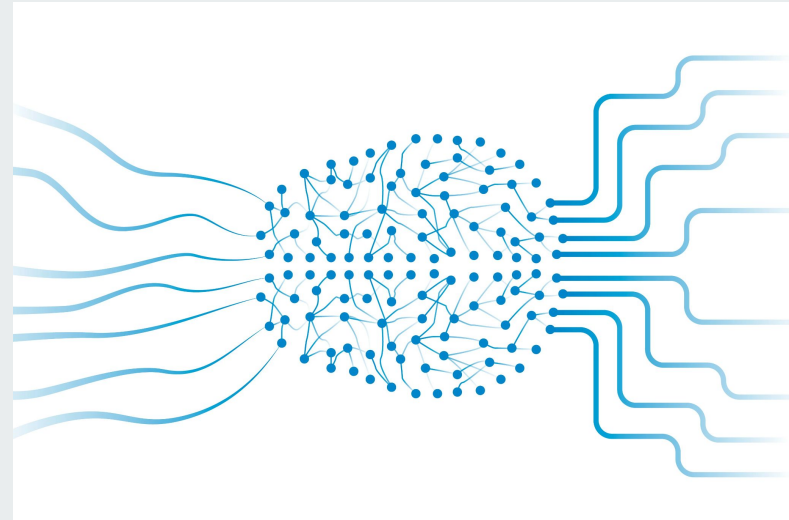
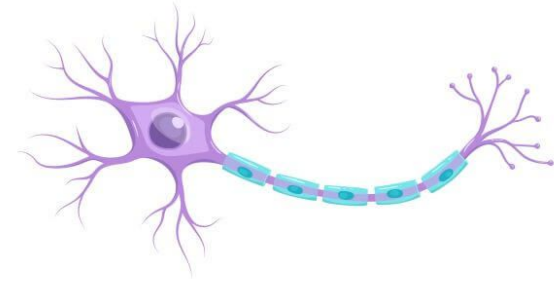

Neural Networks

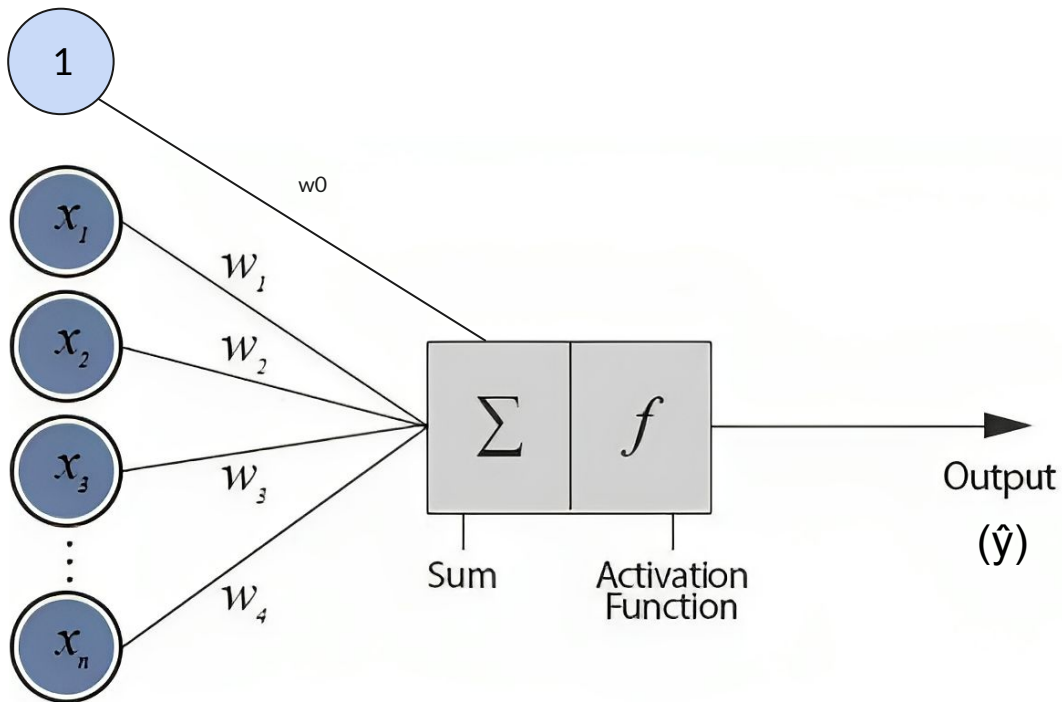


RN – Introducció



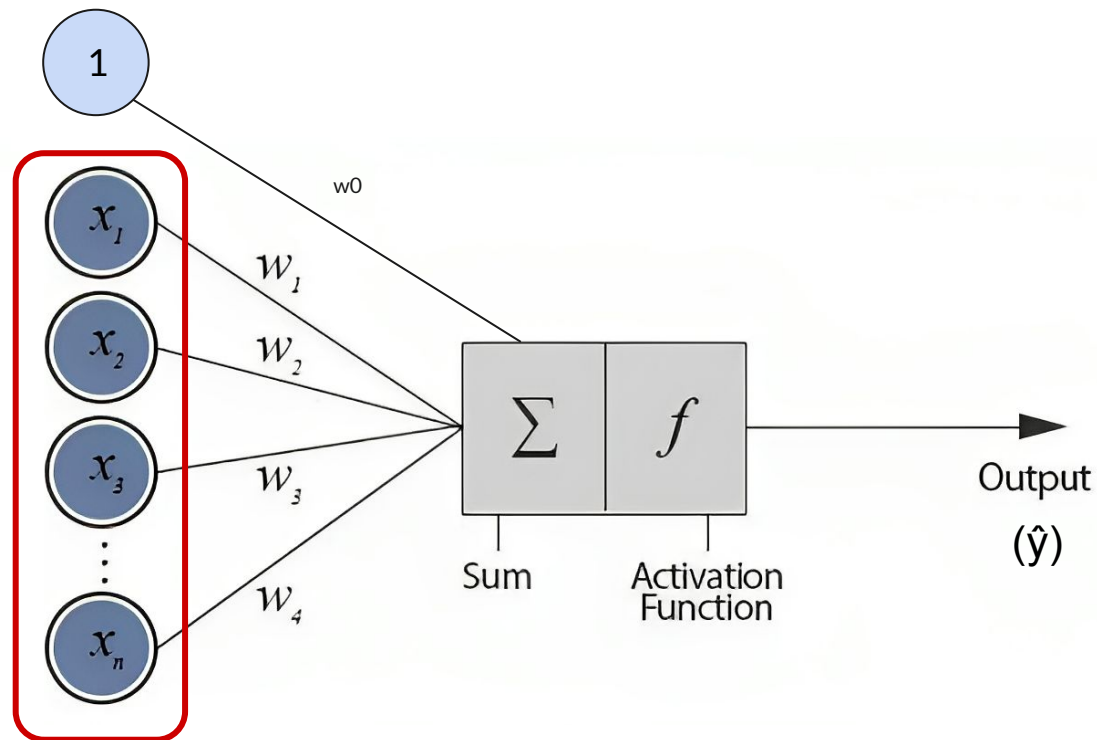
- També conegudes com a Artificial Neural Networks (ANN).
- Mètode de ML usat per a regressió, classificació, etc.
- Inspirat en el cervell humà.
- Molt flexible.
- Normalment no és fàcil d'interpretar.
- Molt famoses avui dia per la seva bona eficàcia en: imatges, text, àudio...

1 Neurona

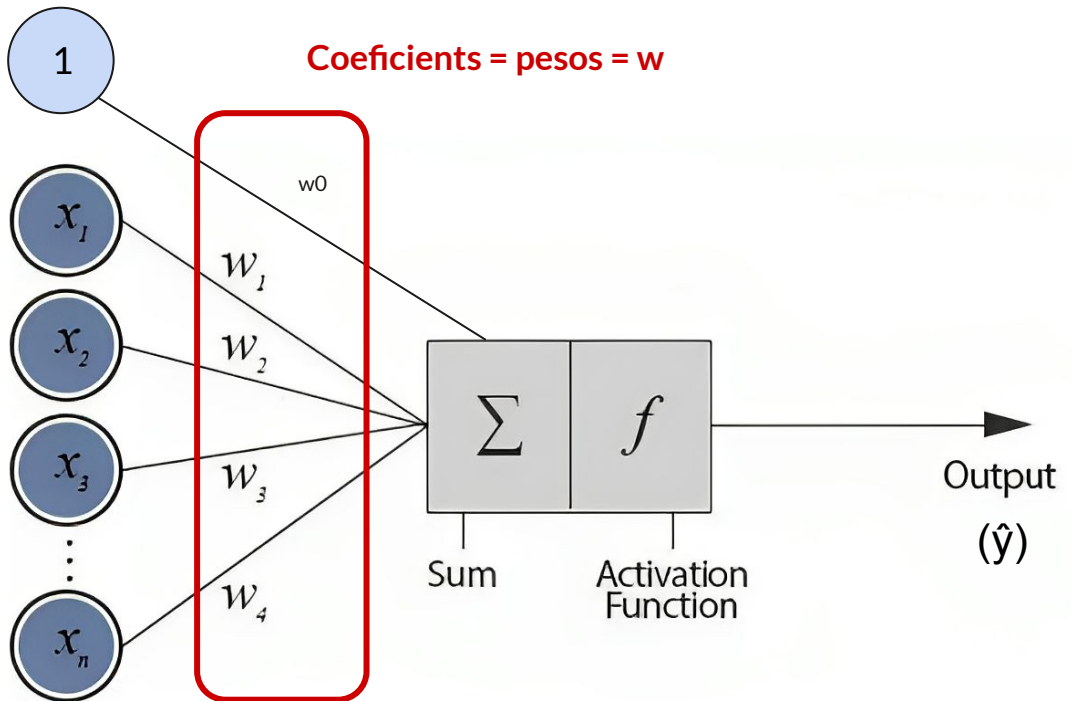


1 Neurona

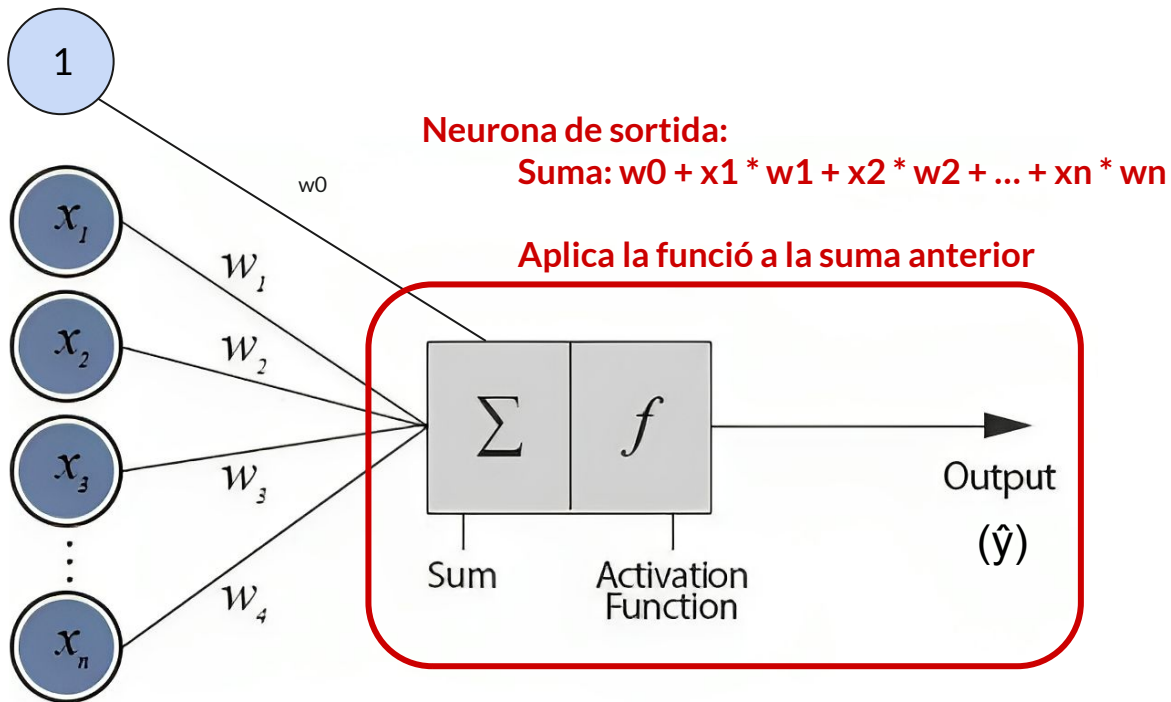
Input = Features



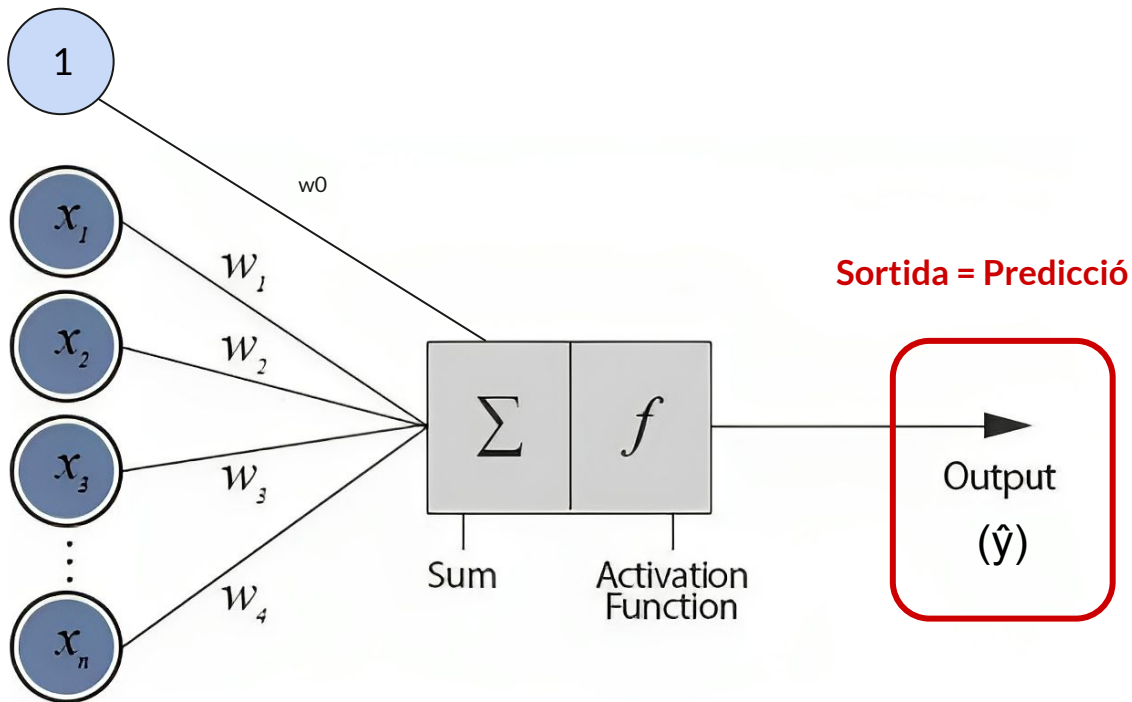
1 Neurona



1 Neurona



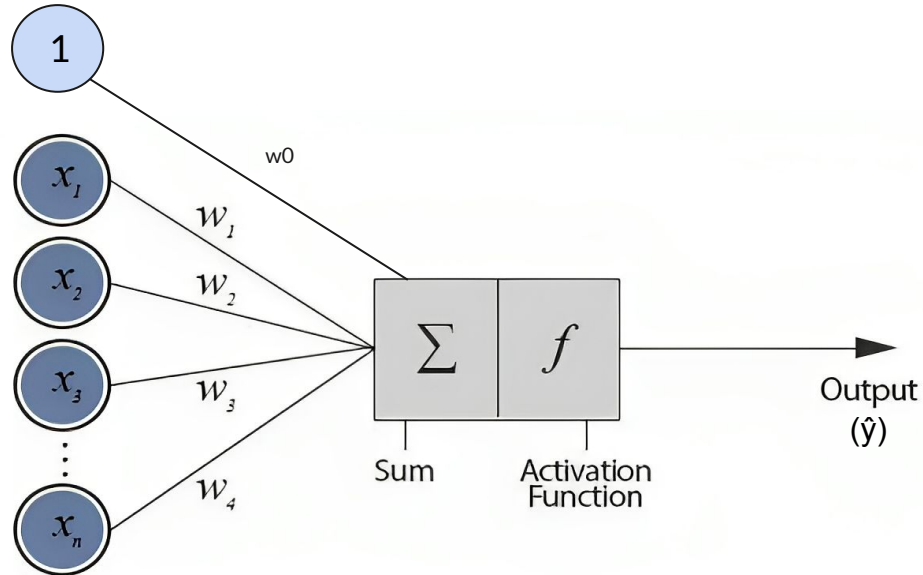
1 Neurona



Entrenament de NN:

1. Normalitzar les característiques (recomanat)

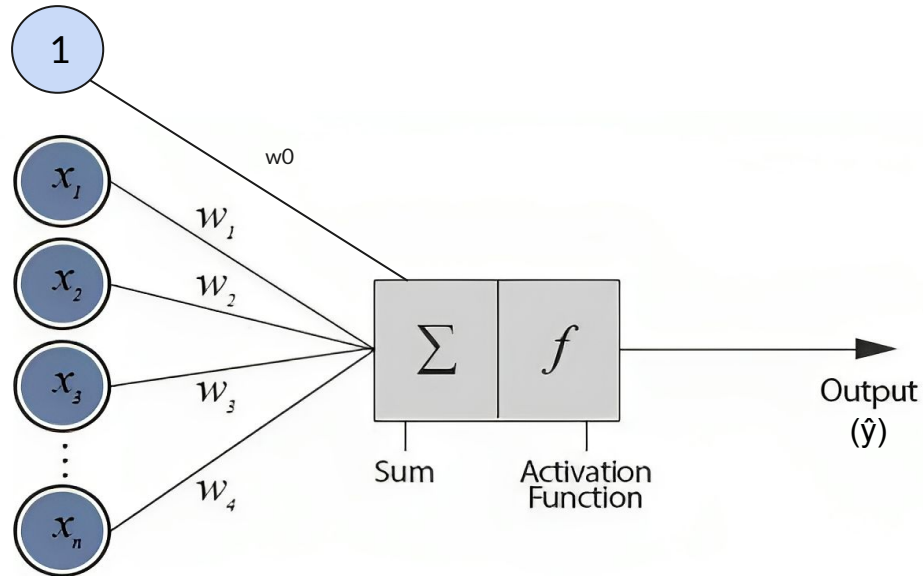
1 Neurona



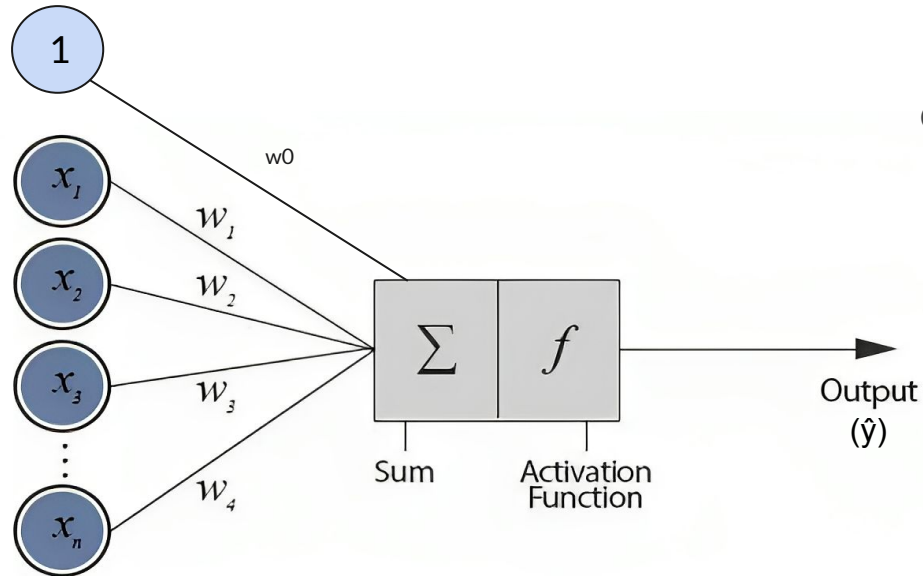
Entrenament de NN:

1. Normalitzar les característiques (recomanat)
2. Inicialitzar pesos w (veure més endavant)

1 Neurona



1 Neurona



Entrenament de NN:

1. **Normalitzar les característiques (recomanat)**
2. **Inicialitzar pesos w (veure més endavant)**
3. Entrenament:

Forward propagation:

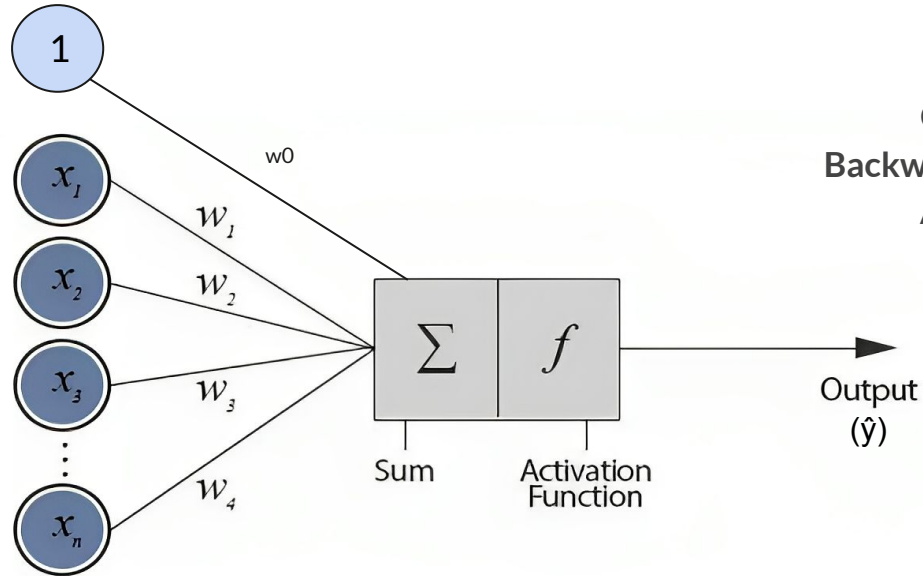
Per a cada fila d'entrada:

Multiplica les característiques pels pesos i suma

Predicció = Funció d'activació de la suma anterior

Calcula la funció de loss (error de predicció)

1 Neurona



Entrenament de NN:

1. **Normalitzar les característiques (recomanat)**
2. **Inicialitzar pesos w (veure més endavant)**
3. Entrenament:

Forward propagation:

Per a cada fila d'entrada:

Multiplica les característiques pels pesos i suma

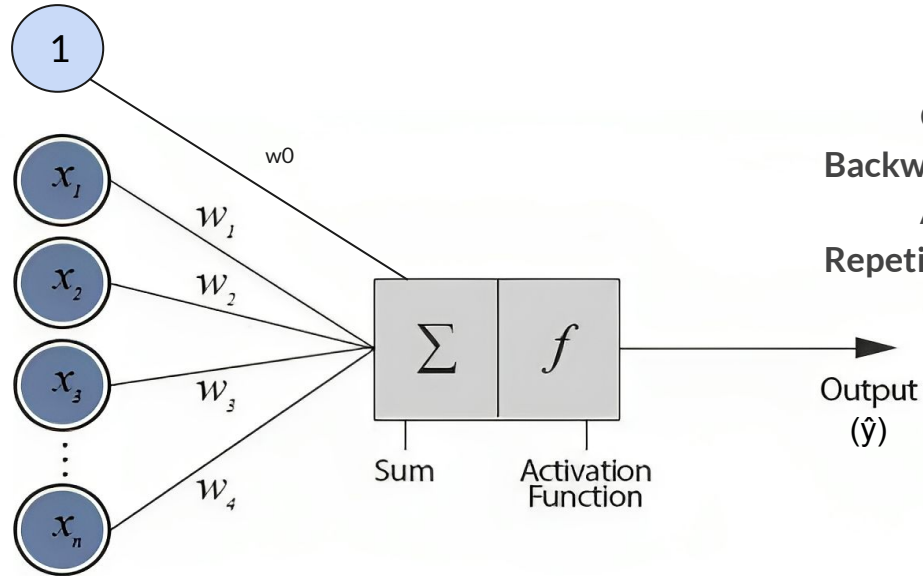
Predicció = Funció d'activació de la suma anterior

Calcula la funció de loss (error de predicció)

Backward propagation:

Actualitza cada pes per optimitzar la funció de loss

1 Neurona



Entrenament de NN:

1. **Normalitzar les característiques (recomanat)**
2. **Inicialitzar pesos w (veure més endavant)**
3. **Entrenament:**

Forward propagation:

Per a cada fila d'entrada:

Multiplica les característiques pels pesos i suma

Predicció = Funció d'activació de la suma anterior

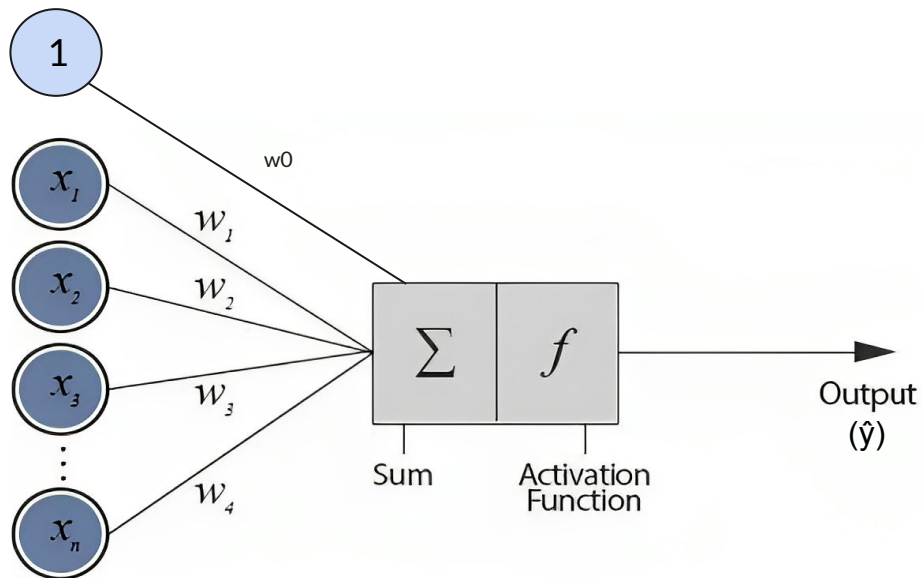
Calcula la funció de loss (error de predicció)

Backward propagation:

Actualitza cada pes per optimitzar la funció de loss

Repetir fins a la convergència o el nombre màxim d'iteracions

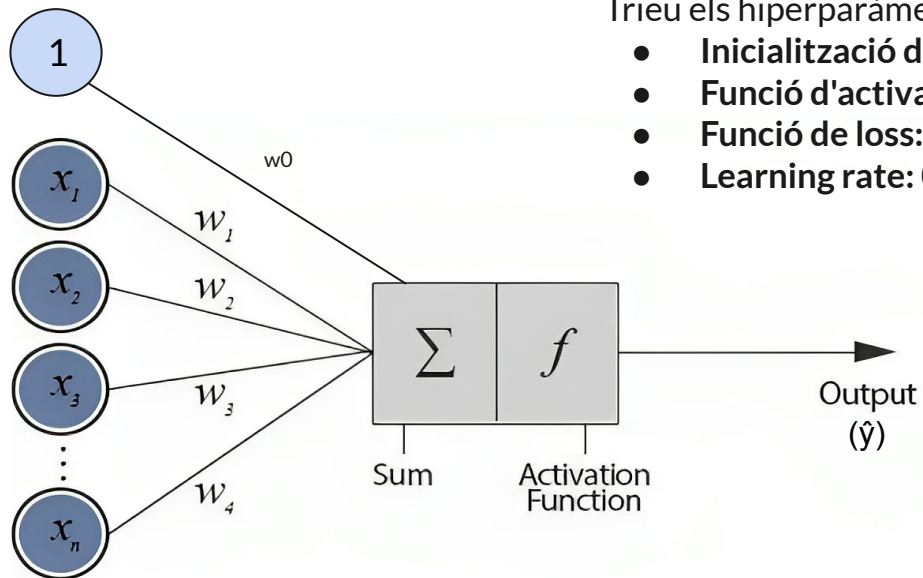
1 Neurona – Exemple



x_1	x_2	x_3	x_4	y
0.1	0.8	0.6	0.8	95
0.3	0.3	0.1	0.2	30

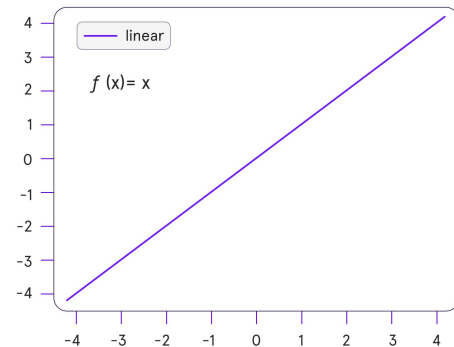
Conjunt de dades amb característiques escalades (regressió).

1 Neurona – Exemple



Trieu els hiperparàmetres:

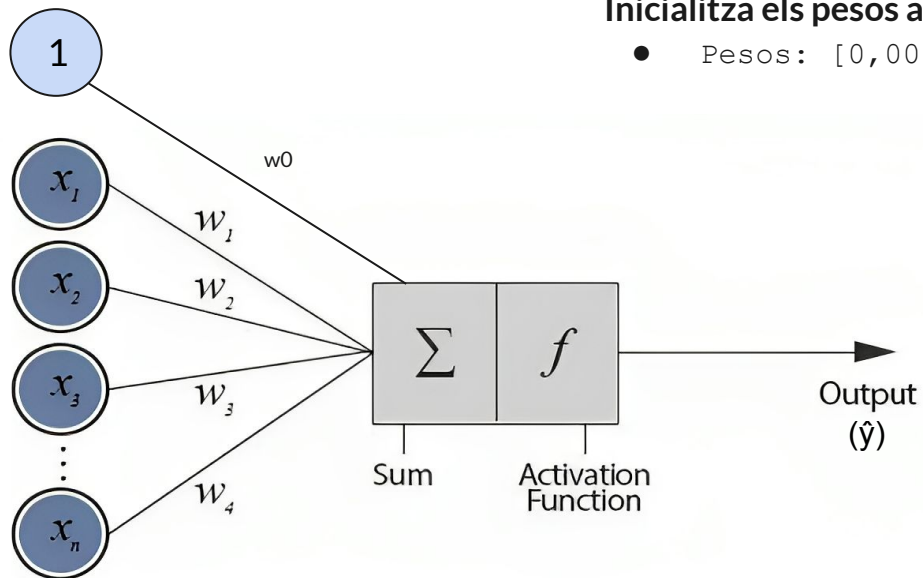
- Inicialització de pesos: Glorot
- Funció d'activació: Lineal
- Funció de loss: MSE
- Learning rate: 0.1



x_1	x_2	x_3	x_4	y
0.1	0.8	0.6	0.8	95
0.3	0.3	0.1	0.2	30

Conjunt de dades amb característiques escalades (regressió).

1 Neurona – Exemple



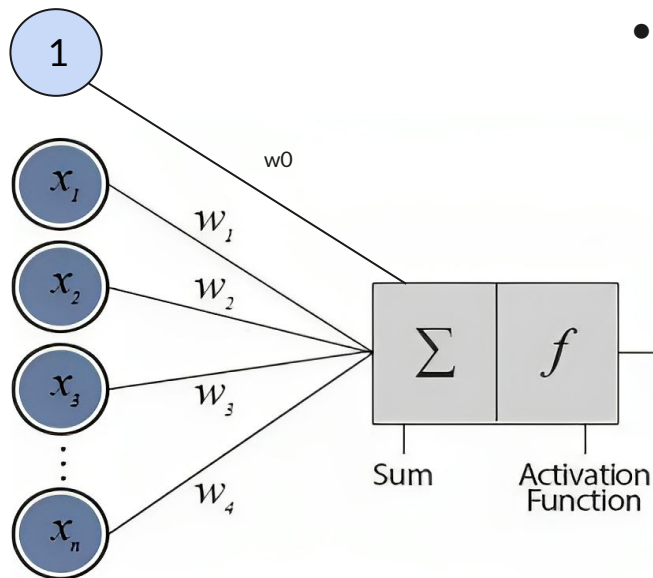
Inicialitza els pesos amb Glorot:

- Pesos: $[0,00, 0,36, 0,72, -0,89, -0,66]$

x_1	x_2	x_3	x_4	y
0.1	0.8	0.6	0.8	95
0.3	0.3	0.1	0.2	30

Conjunt de dades amb característiques escalades (regressió).

1 Neurona – Exemple



Forward propagation:

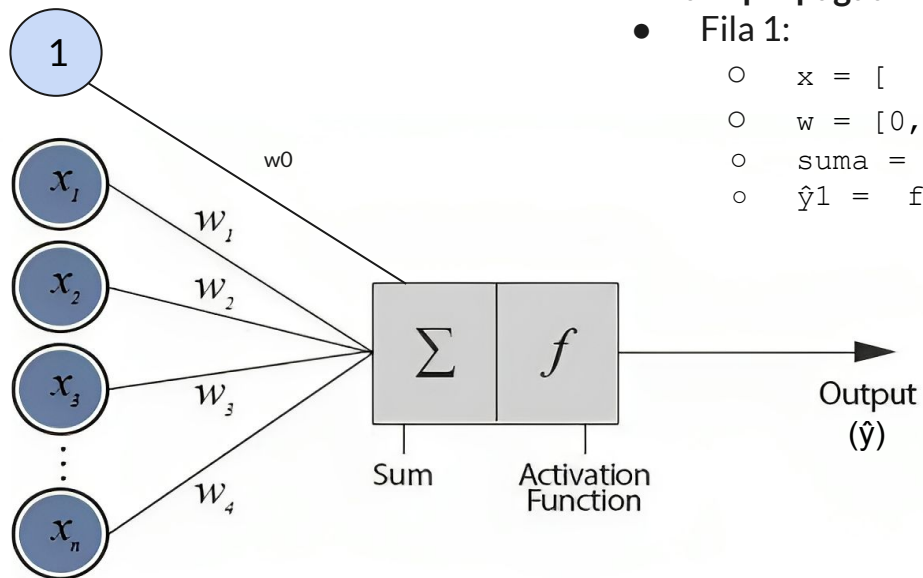
- Fila 0:

- $x = [0,10, 0,80, 0,60, 0,80]$
- $w = [0,00, 0,36, 0,72, -0,89, -0,66]$
- $\text{suma} = 0 + 0,10 * 0,36 + 0,80 * 0,72 + \dots = -0,45$
- $\hat{y}0 = \text{funció_act_linear}(-0,45) = -0,45$

x_1	x_2	x_3	x_4	y
0.1	0.8	0.6	0.8	95
0.3	0.3	0.1	0.2	30

Conjunt de dades amb característiques escalades (regressió).

1 Neurona – Exemple



Forward propagation:

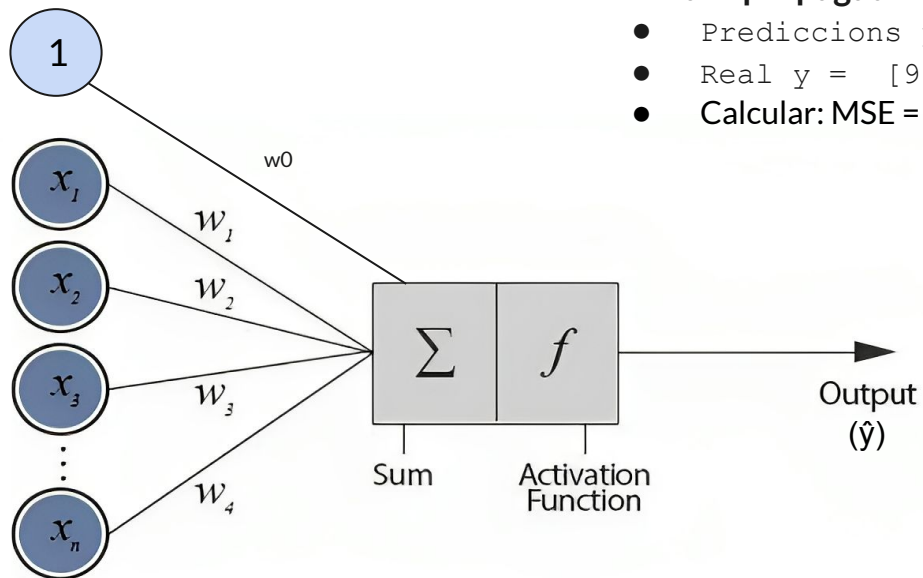
- Fila 1:

- $x = [0,30, 0,30, 0,10, 0,20]$
- $w = [0,00, 0,36, 0,72, -0,89, -0,66]$
- $\text{suma} = 0 + 0,30 * 0,36 + 0,30 * 0,72 + \dots = 0,10$
- $\hat{y}1 = \text{funció_activació_lineal}(0,10) = 0,10$

x_1	x_2	x_3	x_4	y
0.1	0.8	0.6	0.8	95
0.3	0.3	0.1	0.2	30

Conjunt de dades amb característiques escalades (regressió).

1 Neurona – Exemples



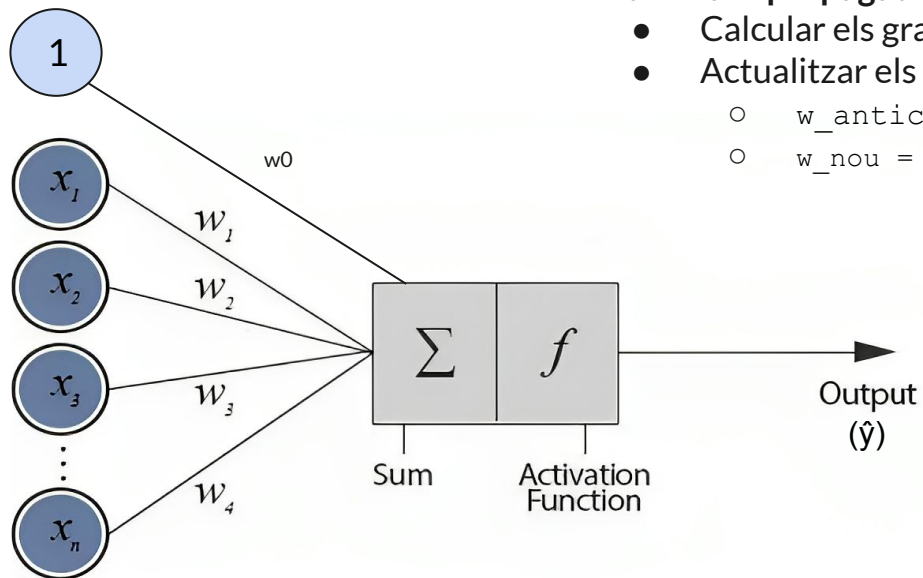
Forward propagation:

- Prediccions $\hat{y}_0 = [-0,45, 0,10]$
- Real $y = [95, 30]$
- Calcular: $MSE = ((95 - 0,45)^2 + (30 - 0,10)^2) / 2 = 5.002$

x_1	x_2	x_3	x_4	y
0.1	0.8	0.6	0.8	95
0.3	0.3	0.1	0.2	30

Conjunt de dades amb característiques escalades (regressió).

1 Neurona – Exemple



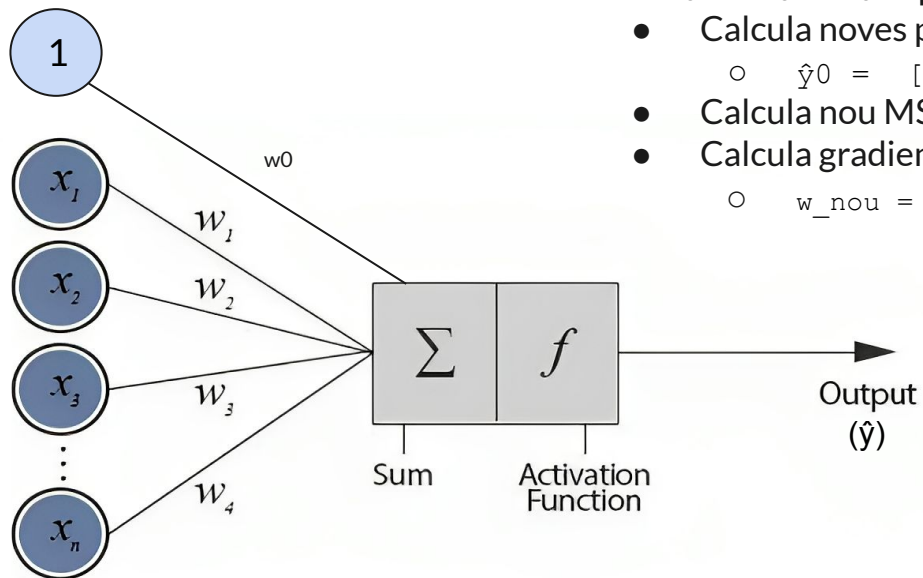
Backward propagation:

- Calcular els gradients dels pesos per minimitzar el MSE.
- Actualitzar els pesos (basat en els gradients i el learning rate):
 - $w_antic = [0,00, 0,36, 0,72, -0,89, -0,66]$
 - $w_nou = [12,54, 3,71, 9,25, 5,73, 7,57]$

x_1	x_2	x_3	x_4	y
0.1	0.8	0.6	0.8	95
0.3	0.3	0.1	0.2	30

Conjunt de dades amb característiques escalades (regressió).

1 Neurona – Exemple



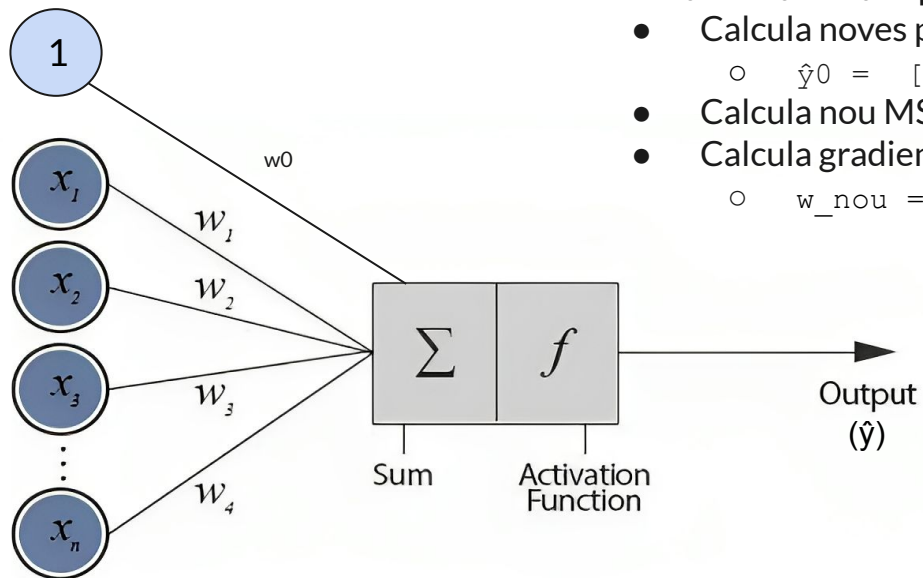
Forward i backward propagation (iteració 2):

- Calcula noves prediccions:
 - $\hat{y}_0 = [29.80, 21.52]$
- Calcula nou MSE = 2.161
- Calcula gradients i actualitza pesos:
 - $w_{\text{nou}} = [13.28, 3.84, 9.80, 6.15, 8.11]$

x_1	x_2	x_3	x_4	y
0.1	0.8	0.6	0.8	95
0.3	0.3	0.1	0.2	30

Conjunt de dades amb característiques escalades (regressió).

1 Neurona – Exemple



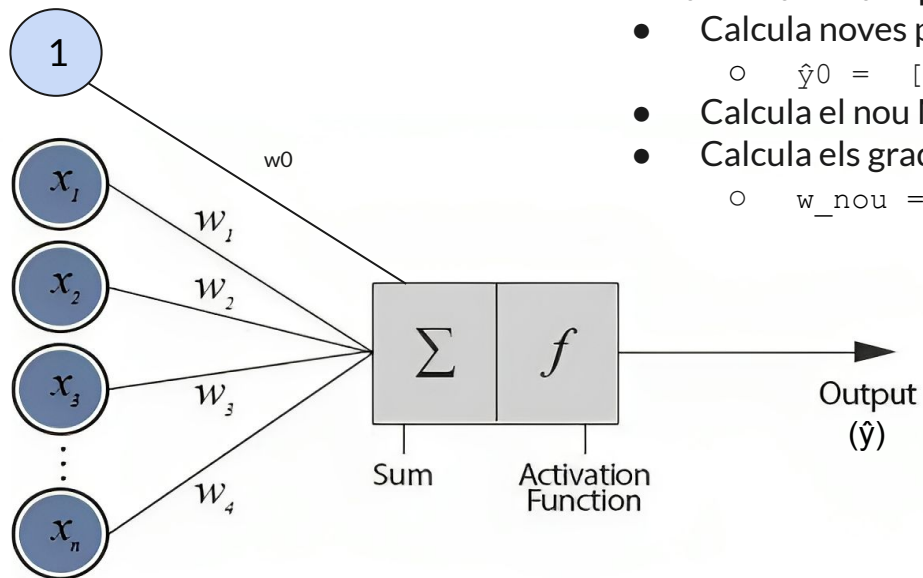
Forward i backward propagation (iteració 3):

- Calcula noves prediccions:
 - $\hat{y}_0 = [31.68, 2.76]$
- Calcula nou MSE = 2.030
- Calcula gradients i actualitza pesos:
 - $w_{\text{nou}} = [13,99, 3,96, 10,33, 6,55, 8,63]$

x_1	x_2	x_3	x_4	y
0.1	0.8	0.6	0.8	95
0.3	0.3	0.1	0.2	30

Conjunt de dades amb característiques escalades (regressió).

1 Neurona – Exemples



Forward i backward propagation (iteració 2.000):

- Calcula noves prediccions:
 - $\hat{y}_0 = [94, 20, 30, 72]$
- Calcula el nou MSE = 0,58
- Calcula els gradients i actualitza els pesos:
 - $w_{\text{nou}} = [23, 25, -22, 30, 35, 81, 22, 92, 38, 47]$

x_1	x_2	x_3	x_4	y
0.1	0.8	0.6	0.8	95
0.3	0.3	0.1	0.2	30

Conjunt de dades amb característiques escalades (regressió).



1 Neurona

- ESPERA! AIXÒ NO ÉS REGRESSIÓ LINEAL?

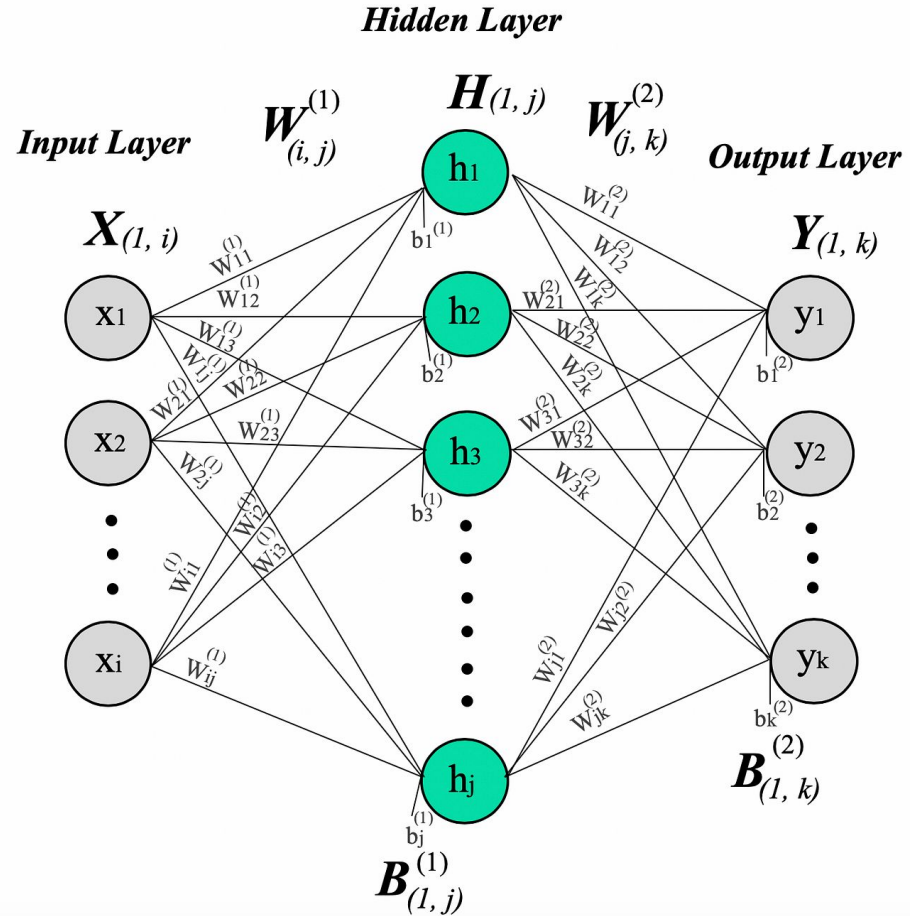


1 Neurona

- ESPERA! AIXÒ NO ÉS REGRESSIÓ LINEAL? SÍ.
- No pot aprendre interaccions no lineals entre característiques.
- No pot aprendre no linealitats complexes entre característiques i objectius.
- S'ha resolt utilitzant capes ocultes amb més neurones i funcions d'activació no lineals.

RN – 3 Capes

- Entrada
- Oculta
- Sortida



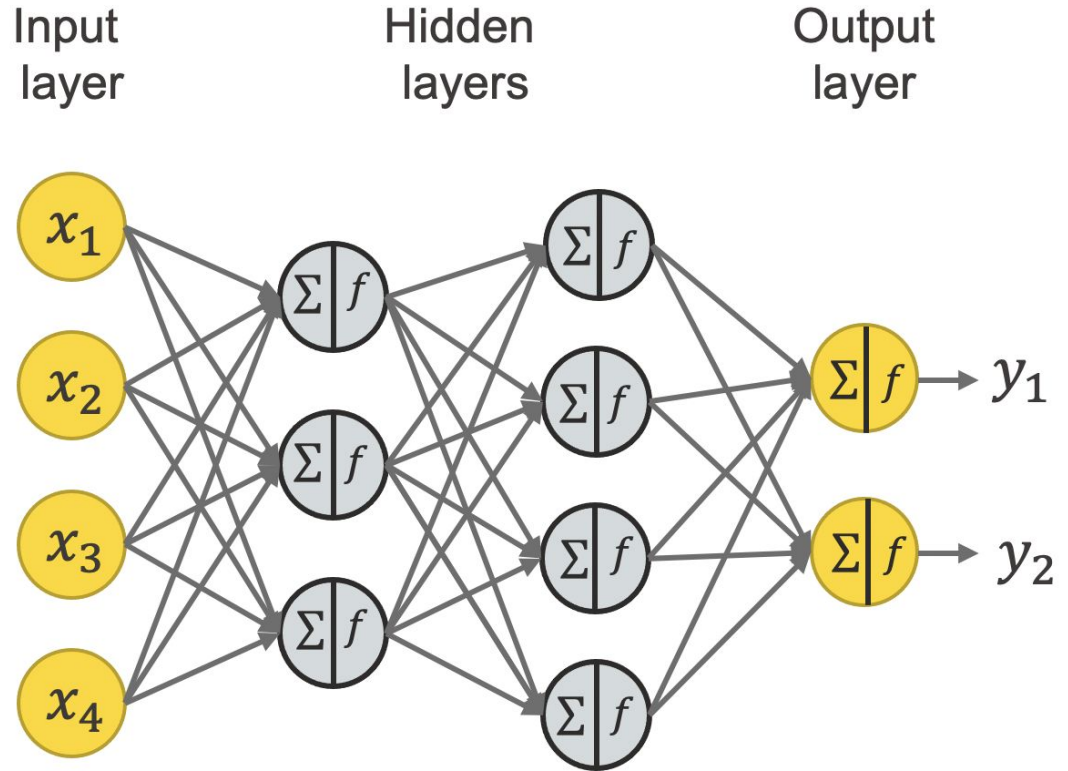


RN – Longituds de capes

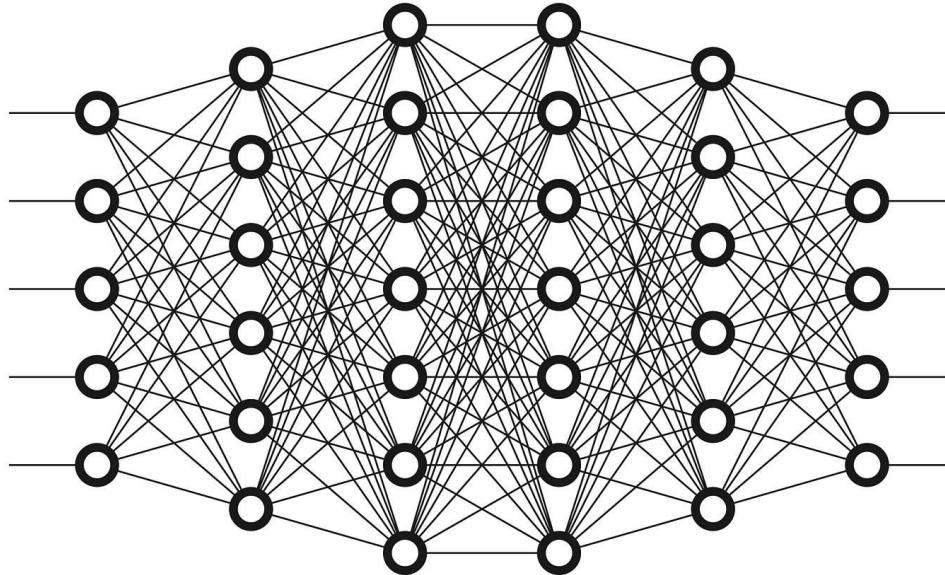
- Entrada: mateixa longitud que el nombre de característiques.
- Oculta: qualsevol longitud que vulguem (vegeu recomanacions més endavant).
- Sortida (vegeu més endavant).

RN – 4 Capes

- Entrada
- Oculta
- Oculta
- Sortida



Xarxa Neuronal Profunda (DNN)



FNN

- Aquest tipus de xarxa neuronal que hem explicat s'anomena:
 - **Xarxa Neuronal Feedforward (FNN).**
- Hi ha més tipus (futur):
 - Xarxes Neuronals Convolucionals (CNNs).
 - Xarxes Neuronals Recurrentes (RNNs).
 - Xarxes Generatives Antagòniques (GANs).
 - Xarxes Transformer.
 - Xarxes Neuronals de Graf (GNNs).
 - Autoencoders.
 - Xarxes Neuronals Basades en Reinforcement Learning.
 - ...

FNN – Hiperparàmetres (per ara)

- Nombre de capes ocultes.

FNN – Hiperparàmetres (de moment)

- Nombre de **capes ocultes**.
- Nombre de **neurones a cada capa**.

FNN – Hiperparàmetres (de moment)

- Nombre de **capes ocultes**.
- Nombre de **neurones a cada capa**.
- Nombre de **neurones a la capa de sortida** → segons el problema (no és un hiperparàmetre).

FNN – Hiperparàmetres (per ara)

- Nombre de **capes ocultes**.
- Nombre de **neurones a cada capa**.
- Nombre de **neurones a la capa de sortida** → segons el problema (no és un hiperparàmetre).
- Algoritme d'**inicialització de pesos**.

FNN – Hiperparàmetres (per ara)

- Nombre de **capes ocultes**.
- Nombre de **neurones a cada capa**.
- Nombre de **neurones a la capa de sortida** → segons el problema (no és un hiperparàmetre).
- Algorisme d'**inicialització de pesos**.
- **Funció d'activació** de cada neurona/capa.

FNN – Hiperparàmetres (per ara)

- Nombre de **capes ocultes**.
- Nombre de **neurones a cada capa**.
- Nombre de **neurones a la capa de sortida** → segons el problema (no és un hiperparàmetre).
- Algorisme d'**inicialització de pesos**.
- **Funció d'activació** de cada neurona/capa.
- Funció de **loss** a optimitzar.

FNN – Hiperparàmetres (per ara)

- Nombre de **capes ocultes**.
- Nombre de **neurones a cada capa**.
- Nombre de **neurones a la capa de sortida** → segons el problema (no és un hiperparàmetre).
- Algorisme d'**inicialització de pesos**.
- **Funció d'activació** de cada neurona/capa.
- Funció de **loss** a optimitzar.
- Algorisme d'optimització dels pesos (**optimitzador**).

FNN – Hiperparàmetres (per ara)

- Nombre de **capes ocultes**.
- Nombre de **neurones a cada capa**.
- Nombre de **neurones a la capa de sortida** → segons el problema (no és un hiperparàmetre).
- Algorisme d'**inicialització de pesos**.
- **Funció d'activació** de cada neurona/capa.
- Funció de **loss** a optimitzar.
- Algorisme d'optimització dels pesos (**optimitzador**).
- **Learning rate**.

FNN – Hiperparàmetres (per ara)

- Nombre de **capes ocultes**.
- Nombre de **neurones a cada capa**.
- Nombre de **neurones a la capa de sortida** → segons el problema (no és un hiperparàmetre).
- Algorisme d'**inicialització de pesos**.
- **Funció d'activació** de cada neurona/capa.
- Funció de **loss** a optimitzar.
- Algorisme d'optimització dels pesos (**optimitzador**).
- **Learning rate**.
- **Mida del mini-batch**.

FNN – Hiperparàmetres (per ara)

- Nombre de **capes ocultes**.
- Nombre de **neurones a cada capa**.
- Nombre de **neurones a la capa de sortida** → segons el problema (no és un hiperparàmetre).
- Algorisme d'**inicialització de pesos**.
- **Funció d'activació de cada neurona/capa**.
- Funció de **loss** a optimitzar.
- Algorisme d'optimització dels pesos (**optimitzador**).
- **Learning rate**.
- **Mida del mini-batch**.
- Nombre d'iteracions de propagació forward/backward (**epochs**).



RN – Teorema d'Aproximació Universal

- Una xarxa feedforward amb:
 - Una única capa oculta.
 - Un nombre suficient de neurones.
 - Funcions d'activació apropiades (moltes comunes funcionen, e.g., sigmoid).
- **Pot aproximar qualsevol funció contínua de n dimensions amb la precisió desitjada.**



RN – Teorema d'Aproximació Universal

- Inconvenients:
 - Tot i que una única capa POT aproximar qualsevol funció...
 - Pot necessitar un nombre de neurones exponencialment gran.
 - Això el fa computacionalment poc pràctic per a funcions complexes.
 - El teorema només ens indica que tal xarxa EXISTEIX, no com trobar-la.

RN – Teorema d'Aproximació Universal

- Per què les xarxes profundes funcionen millor:
 - Sovint poden representar la mateixa funció amb moltes menys neurones.
 - Aprenen característiques jeràrquiques:
 - Les capes inicials aprenen característiques simples i de baix nivell. Per exemple:
 - A les imatges: vores, cantonades, textures...
 - En text: patrons de paraules individuals.
 - Les capes successives aprenen característiques cada cop més complexes i abstractes combinant característiques de nivell inferior. Per exemple:
 - En imatges: poden detectar formes i objectes específics...
 - En text: poden comprendre relacions semàntiques i context.



FNN – Fins a quin punt profund? Quins hiperparàmetres triar?

- Totalment dependent del context...
 - Tasques a resoldre.
 - Mida del conjunt de dades i qualitat de les dades.
 - Restriccions computacionals.
 - ...



Construcció de xarxes neuronals feedforward – Recomanacions

- Capa d'entrada:
 - Fàcil! El nombre de neurones d'entrada és igual al nombre de característiques.



Construcció de FNNs – Recomanacions

- Nombre de capes ocultes:
 - Una o dues per a problemes simples.
 - Més profundes per a dades o tasques més complexes.



Construcció de FNNs – Recomendacions

- Nombre de capes ocultes:
 - Una o dues per a problemes senzills.
 - Més profunda per a dades o tasques més complexes.
- Neurones per capa:

Construcció de FNNs – Recomanacions

- Nombre de capes ocultes:
 - Una o dues per a problemes senzills.
 - Més profundes per a dades o tasques més complexes.
- Neurones per capa:
 - Per simplicitat en problemes fàcils:
 - El mateix nombre de neurones per capa.
 - Nombre similar a l'entrada (potser una mica menys, potser una mica més).



Construint FNNs – Nombre de Neurones

- Més neurones poden representar patrons més complexos.
- Més neurones són més lentes d'entrenar.
- Un excés de neurones pot conduir a un sobreajustament.
- Poques neurones poden comportar un subajustament.

Construcció de FNNs – Recomanacions

- Capa de sortida:
 - Sovint determinada pel problema.
 - Regressió: 1 neurona amb, normalment, funció lineal.
 - Classificació:
 - Binària: 1 neurona amb una funció sigmoide.
 - Multiclasse: N neurones (per a N classes) amb:
 - Softmax si és d'etiqueta única.
 - Sigmoide si és d'etiquetes múltiples.



NN – Inicialització dels coeficients

- Per què necessitem la inicialització
 - Si tots els pesos comencen iguals, totes les neurones computarien la mateixa sortida.
 - Això significa que obtindrien els mateixos gradients i aprendrien les mateixes coses.
 - Cal "trencar la simetria" entre les neurones.
 - També cal evitar alguns problemes (per exemple, gradients que desapareixen o exploten → més endavant).



NN – Algorismes d'inicialització de coeficients

- Xavier / Glorot (també conegut com a Glorot uniforme)
- He
- Normal estàndard
- LeCun
- Ortogonal

NN – Algoritmes d'inicialització de coeficients

- Xavier / Glorot (també conegut com a Glorot uniforme)
- He
- Normal estàndard
- LeCun
- Ortogonal

No us preocupeu per això
ara mateix... useu la
configuració per defecte.

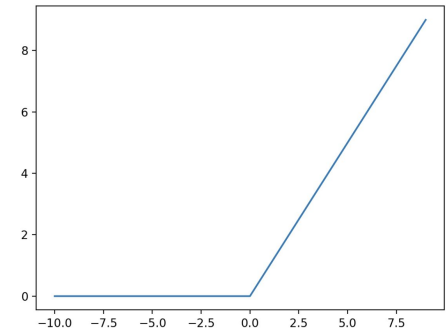
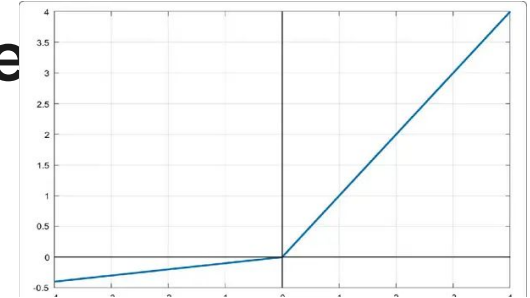
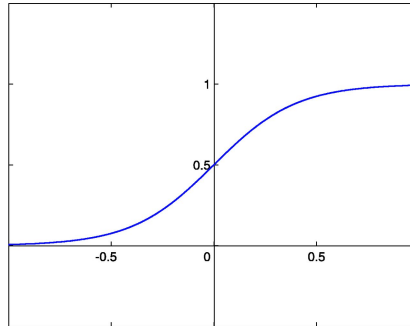


FNN – Funció d'activació

- Funcions d'activació lineals per aprendre patrons lineals entre característiques i objectiu.
- Funcions d'activació no lineals per aprendre no linealitats entre característiques i objectiu.

FNN – Funcions d'activació comune

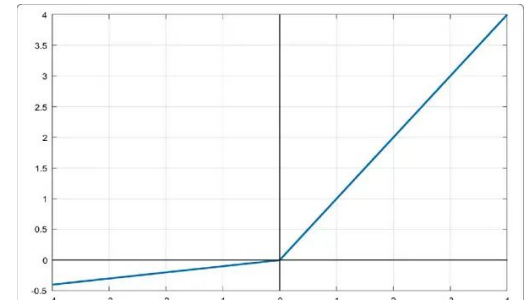
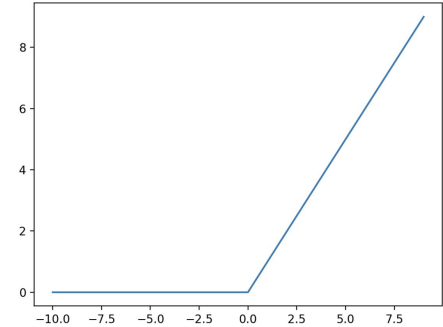
- Lineal
- Sigmoide
- Tanh
- ReLU
- Leaky ReLU
- ELU
- GELU
- Swish
- Softmax



FNN – Funcions d'Activació Comunes

- Lineal
- Sigmoidal
- Tanh
- ReLU
- Leaky ReLU
- ELU
- GELU
- Swish
- Softmax

Capes
ocultes



FNN – Funcions d'activació comunes

- Lineal
- Sigmoide
- Tanh
- ReLU
- Leaky ReLU
- ELU
- GELU
- Swish
- Softmax

Capa de
sortida

Capa de
sortida



FNN – Pèrdua

- Regressió:
 - MSE
 - MAE
 - Pèrdua Huber (robusta als valors atípics).

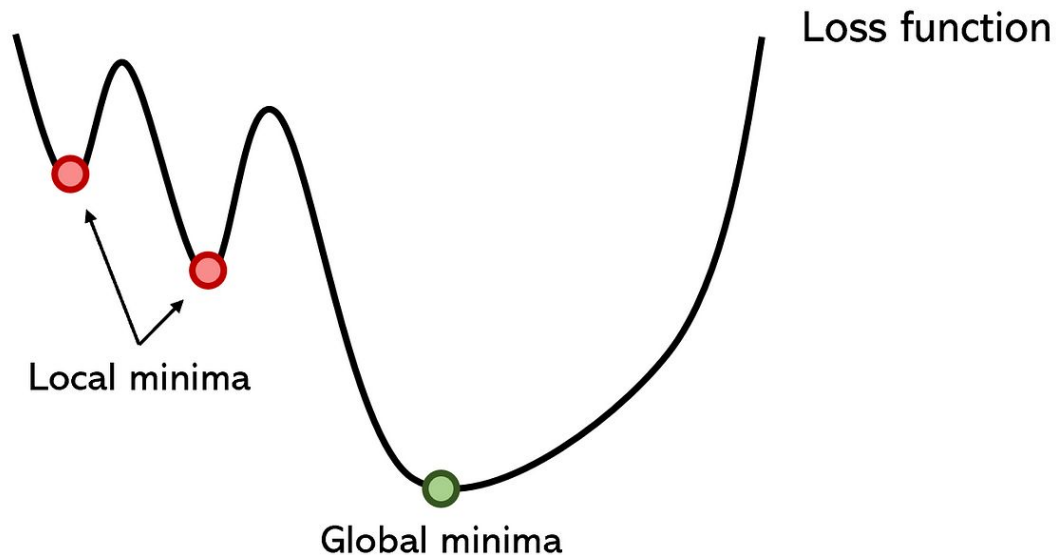


Pèrdua FNN

- Classificació:
 - Binary cross-entropy (igual que la logloss).
 - Sparse categorical cross-entropy per a objectius enters multiclasse.
 - Categorical cross-entropy per a objectius multiclasse codificats en one-hot.

FNN – Optimitzant la pèrdua

- No convex.
- Risc de quedar-se encallat.
- Bona taxa d'aprenentatge.
 - Optimizers:
 - Adam
 - RMSprop
 - AdaGrad
 - ...



FNN – Optimizadors

**STOCHASTIC
GRADIENT
DESCENT**



**SGD WITH
MOMENTUM**



RMSPROP



ADAM





FNN – Taxa d'aprenentatge

- Pot ser difícil d'endevinar...
- Prova'n unes quantes i mira quina funciona millor: 0,1, 0,01, 0,001, etc.

FNN – Mida del mini-batch (batch size)

- Potències de 2 (p. ex., 32, 64, 128) s'alineen amb architectures GPU i TPU.
- Batches més petits (p. ex., 8) poden provocar massa soroll.
- Batches més grans (p. ex., 256) tenen risc de mala generalització.
- S'ha observat que la **mida del batch 32 ofereix un bon equilibri** entre aleatorietat durant l'entrenament i velocitat de convergència.
 - Per tant, 32 pot ser un bon punt de partida.
 - Si no funciona correctament, experimenta amb altres mides.

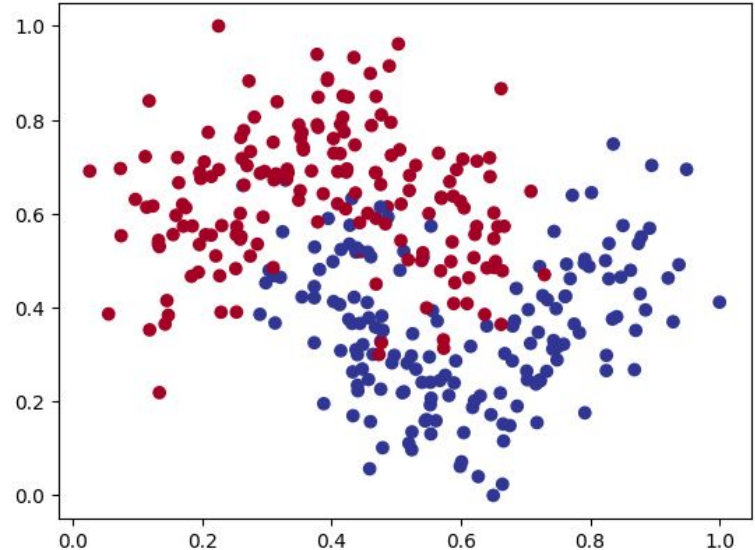


FNN – Epochs

- Una epoch consisteix en una passada per totes les mostres d'entrenament.
- Quan es separa en mini-batches, una epoch itera sobre tots els mini-batches.
- Les NNs requereixen múltiples epochs per a una bona optimització dels pesos.
- **Massa epochs** pot portar a **overfitting**.
- **Pocs epochs** pot resultar en underfitting.
- Es pot aplicar early stopping.

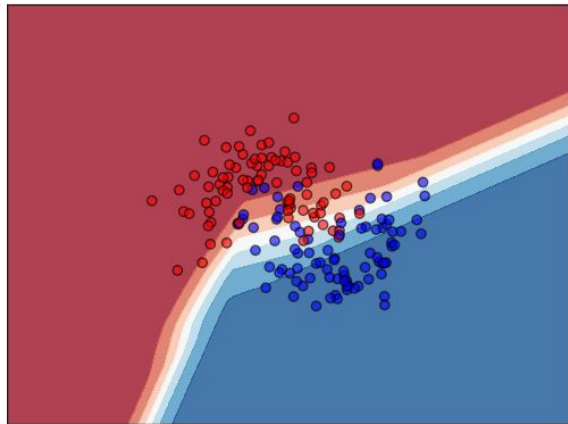
FNN – Examples

- 4 capes:
 - 2 neurones d'entrada (2 features)
 - 16 neurones ReLU ocultes
 - 8 neurones ReLU ocultes
 - 2 neurones de sortida amb softmax
- Optimitzador: Adam(0,01)
- Loss: entropia creuada categòrica
- Batch size: 32

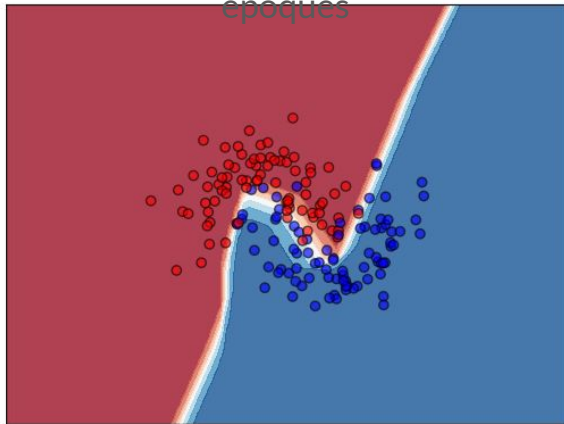


FNN – Exemple

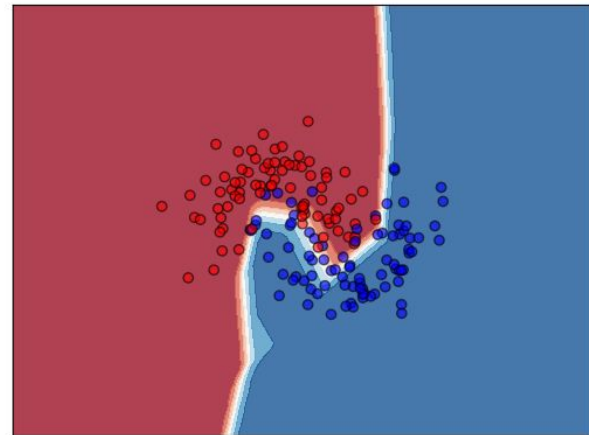
50 époques



200
époques



2.000 époques



FNN - Propagació cap endavant

L'entrada a una capa es calcula com:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

Les activacions es calculen com:

$$a^{(l)} = \sigma(z^{(l)})$$

Funció d'activació

on:

- $z^{(l)}$: Pre-activation vector for layer l (shape n_l).
- $W^{(l)}$: Weight matrix for layer l (shape $n_l \times n_{l-1}$).
- $a^{(l-1)}$: Activations from the previous layer (shape n_{l-1}).
- $b^{(l)}$: Bias vector for layer l (shape n_l).



RN – Llibreries

- Keras
- Tensorflow / Keras
- Pytorch
- `sklearn.neural_network.MLPClassifier` o `MLPRegressor`

Resum de Conceptes

- Xarxa Neuronal (NN).
- Xarxa Neuronal Profunda (DNN).
- Xarxa Neuronal Feedforward (FNN).
- Pesos (similars als coeficients de la regressió lineal).
- Funcions d'activació.
- Capas d'Entrada, Ocultes, de Sortida.
- Propagació forward/backward.
- Optimitzadors: Adam, RMSprop, AdaGrad...
- Epoch.



NN Playground

- <https://playground.tensorflow.org>